

Agile & CI/IaC

Agile Delivery als Steuerungssystem –
Scrum, Kanban, CI/CD und Infrastructure as Code für sichere Änderungen.

Lehrskript – Tag 9 von 16

Lernpfad:
Wissen → Anwendung → Management

Dozent:
Can Siebert

Bearbeitungsdauer:
1 Kurstag

Format:
Theorie · Fallstudie · Coaching

Lernziele dieses Tages

Modelle, Mining und Predictive Analytics sind die analytischen Schichten – sie zeigen, was passiert. Heute treten wir auf die *Liefer-Ebene*: Wie werden Prozess- und IT-Änderungen *verantwortet, getaktet und sicher in Produktion gebracht*? Vier Bausteine machen das möglich – Scrum als Sprint-Framework, Kanban als Flow-Methode, Continuous Integration als Beschleuniger und Infrastructure as Code als versionierbares Fundament.

L1 – Wissen (Reproduktion und Einordnung)

- Scrum-Bausteine kennen: Rollen (Product Owner, Scrum Master, Developers), Events, Artefakte, Definition of Done.
- Kanban-Prinzipien einordnen: Visualisieren, WIP-Limits, Pull-System, Flow-Metriken.
- CI-Pipeline und ihre typischen Stufen kennen: Build, Tests, Security/Quality Checks, Deploy.
- Infrastructure-as-Code-Grundlagen benennen: Code statt Click, Versionierung, Reviews, Audit-Trail.

L2 – Anwendung (Transfer auf konkrete Situationen)

- Eine fundierte Entscheidung Scrum / Kanban / Hybrid in einem konkreten Setting treffen.
- Ein Sprint- oder Kanban-Board mit Rollen, Policies und Replenishment-Mechanik entwerfen.
- Eine minimale CI-Pipeline mit zwei nicht verhandelbaren Quality Gates konstruieren.
- IaC-Policy mit Review-Pflicht, Freigabe und Audit-Trail formulieren.

L3 – Management (Entscheidung und Verantwortung)

- DORA-Metriken (Lead Time, Deployment Frequency, Change Failure Rate, MTTR) als Steuerungssprache nutzen.
- Zentrale (Gates, Security, IaC-Standards) vs. dezentrale Entscheidungen (Team Backlog) bewusst trennen.
- “Fast vs. Safe”-Entscheidungen unter Zielkonflikt schriftlich begründen.
- Anti-Gaming-Mechanismen für Flow-Kennzahlen einbauen.

Roter Faden

Agile ist kein Synonym für “schneller”. Es ist ein Synonym für “häufig liefern, kurz feedbacken, früh lernen.” CI/IaC sind keine Tools, sondern *Vereinbarungen*: jeder Build prüfbar, jede Änderung versioniert, jedes Deployment rückbaubar. Senior-Praxis verbindet: *Flow plus Gates plus Ownership = sichere Geschwindigkeit.*

1. Einstieg: Veränderung als gesteuertes System

Jedes BPM-, Digital- oder Transformations-Programm hat eine Lieferschicht. Hier werden Prozess- und IT-Änderungen entworfen, gebaut, getestet, freigegeben, ausgerollt – und im Idealfall messbar verbessert. Wer diese Schicht ignoriert, hat zwar Modelle und Pipelines, aber keine Lieferfähigkeit. (Beck et al., 2001; Forsgren et al., 2018)

1.1 Symptome einer fehlenden Lieferarchitektur

- **“Release Friday Fear”.** Niemand traut sich, am Freitag noch zu deployen – aus Angst, das Wochenende durchzukämpfen.
- **Sprint Theater.** Scrum-Events laufen, aber jede Iteration liefert das Gleiche: Tickets schieben, keine Outcomes.
- **Kanban-Wildwuchs.** Boards mit 30 Tickets in “In Progress”; WIP-Limits nicht durchgesetzt; Pull-System nur auf dem Papier.
- **Deploys per Klick.** Niemand weiß, was im letzten Release wirklich geändert wurde; Rollback ist Glücksspiel.

1.2 Die vier Bausteine sicherer Lieferung

1. **Scrum** – Sprint-Taktung, Rollen, Events, Definition of Done.
2. **Kanban** – Flow, WIP-Limits, Pull-System, Lead Time messen.
3. **CI/CD** – automatisierte Build/Test/Deploy-Pipelines mit Quality Gates.
4. **Infrastructure as Code** – versionierte, reviewbare, reproduzierbare Infrastruktur.

Diese vier Bausteine sind keine isolierten Tools, sondern Bausteine eines *Steuerungssystems*: Taktung, Flow, Sicherheit, Rückbaubarkeit.

Eselsbrücke: S-K-C-I

Vier Bausteine sicherer Liefer-Architektur:

Scrum – Taktung für planbare Increments.

Kanban – Flow für kontinuierliche Lieferung.

CI/CD – automatisiertes Bauen, Testen, Deployen.

IaC – Infrastruktur versionieren, reviewen, rückbauen.

Wer alle vier zusammen denkt, baut sichere Geschwindigkeit – statt schnellem Chaos.

2. Scrum: Sprints, Rollen, Events, Artefakte

Scrum ist seit den 1990ern das verbreitetste agile Framework. Der aktuelle Scrum Guide (Schwaber & Sutherland, 2020) reduziert es auf wenige, klar definierte Rollen, Events und Artefakte.

(Schwaber & Sutherland, 2020; Highsmith, 2009)

2.1 Die drei Rollen

- **Product Owner.** Verantwortet das Produkt-Backlog, priorisiert nach Wert, entscheidet über Inhalte. *Eine Person* – nicht ein Komitee.
- **Scrum Master.** Coacht das Team, beseitigt Impediments, stellt sicher, dass Scrum gelebt wird (kein klassischer Projektmanager).
- **Developers.** Cross-funktionales Team, das das Increment liefert (umfasst Entwickler, Tes-

ter, Designer, Architekten).

2.2 Die vier Events

- **Sprint Planning.** Zu Beginn des Sprints. Team wählt aus Backlog, definiert Sprint Goal und Sprint Backlog. Dauer typischerweise 2–4 h für einen 2-Wochen-Sprint.
- **Daily Scrum.** Tägliche 15-Minuten-Synchronisation. Nicht Status für Manager, sondern Team-Selbstkoordination.
- **Sprint Review.** Am Ende des Sprints. Increment wird Stakeholdern vorgestellt, Feedback fließt ins Backlog.
- **Sprint Retrospective.** Team-Reflexion: Was lief gut, was nicht, welche eine Sache verbessern wir?

2.3 Die drei Artefakte

- **Product Backlog.** Geordnete Liste aller bekannten Anforderungen. Lebt, wird ständig gepflegt.
- **Sprint Backlog.** Auswahl des Sprints + Plan zur Lieferung.
- **Increment.** Nutzbares Inkrement am Sprint-Ende. Muss die **Definition of Done (DoD)** erfüllen.

2.4 Definition of Done

Die DoD ist eine ehrliche Liste prüfbarer Kriterien, ab denen ein Backlog Item “fertig” ist. Sie ist team- und produktabhängig. Typische DoD-Punkte:

- Code-Reviews vollständig.
- Unit-Tests grün.
- Integration-Tests grün.
- Security-Scan ohne kritische Findings.
- Dokumentation aktualisiert.
- In Staging deployed.
- Vom Product Owner abgenommen.

2.5 Nutzen und Risiken

- **Nutzen:** planbare Taktung, regelmäßiges Feedback, klare Priorisierung.
- **Risiken:** *Cargo-Cult Scrum* (Events ohne Outcome), Sprint als Wasserfall-Mini-Projekt, Backlog als Wunschzettel statt Priorität.

Scrum-Disziplin

Ein Daily ohne Selbst-Koordination ist Status-Theater. Ein Sprint Review ohne Stakeholder-Feedback ist Demo ohne Schluss. Eine Retrospective ohne konkreten Verbesserungs-Schritt ist Therapie. Scrum trägt nur, wenn die Events *Wirkung* statt *Ritual* sind.

3. Kanban: Flow, WIP-Limits, Pull-System

Kanban stammt aus der Lean-Tradition und wurde von David J. Anderson für Wissensarbeit adaptiert. Im Gegensatz zu Scrum hat Kanban keine fixe Iteration – es ist ein *kontinuierlicher*

Flow. (Anderson, 2010; Reinertsen, 2009)

3.1 Die fünf Kanban-Prinzipien

1. **Visualisieren.** Alle Arbeit auf einem Board sichtbar machen.
2. **WIP limitieren.** “Work In Progress” pro Spalte begrenzen.
3. **Flow messen.** Lead Time, Cycle Time, Throughput.
4. **Policies explizit machen.** Was ist die Definition pro Spalte, was ist Done-Kriterium?
5. **Kontinuierlich verbessern.** Kaizen-Mentalität, datengetriebene Optimierung.

3.2 Board-Aufbau

Ein typisches Board hat Spalten wie:

- *Backlog* – gepflegt, priorisiert.
- *Ready* – definiert, ziehbar.
- *In Progress* – WIP limitiert.
- *Review* – Code Review, Test.
- *Done* – abgeschlossen, geht in Cycle-Time-Messung ein.

Jede Spalte mit **Eintrittskriterien** (was muss erfüllt sein, um in die Spalte zu ziehen) und **Austrittskriterien** (was muss erfüllt sein, um weiterzugehen).

3.3 WIP-Limits

WIP-Limits sind die wichtigste Stellschraube im Kanban. Sie verhindern Multitasking, machen Engpässe sichtbar und reduzieren Lead Time.

- Faustregel: *Anzahl Team-Mitglieder* $\times$ 1 bis 1,5 pro Spalte.
- Bei Überschreitung: Stop the Line. Niemand neues anfangen, bevor blockiertes Item gelöst ist.
- Tägliches Stand-up: “Wo blockiert es, was tut der Schwarm?”.

3.4 Flow-Metriken

- **Lead Time.** Zeit von “in Backlog” bis “Done”. Geschäftsrelevant.
- **Cycle Time.** Zeit von “In Progress” bis “Done”. Teamintern.
- **Throughput.** Anzahl Items pro Zeiteinheit.
- **Aging WIP.** Wie lange ist ein Item schon “In Progress”? Frühwarnsignal für Stuck-Items.

3.5 Replenishment-Meeting

Wenn “Ready” leerläuft, muss nachgefüllt werden – ohne Sprint-Rhythmus. Pragmatisch: ein wöchentliches **Replenishment-Meeting** mit Product Owner, Lead, Team. Auswahl der nächsten Items, Aufnahme in “Ready”.

WIP-Limit als Selbstdisziplin

WIP-Limits funktionieren nur, wenn das Team sie *respektiert* – sonst werden sie zur Dekoration. Ein Team, das WIP-Limits konsequent durchhält, wird kurzfristig langsamer wirken (weniger gleichzeitige Tickets), aber mittelfristig deutlich schneller liefern (kürzere Lead Time, weniger Rework).

3.6 Nutzen und Risiken

- **Nutzen:** Stabilität, Durchsatz, weniger Multitasking, klare Engpässe.
- **Risiken:** ohne explizite Policies und Replenishment wird Kanban zu “Trello-Board mit Spalten” – ohne Steuerungseffekt.

4. Scrum, Kanban oder Hybrid?

Es gibt keine universelle Antwort. Die Wahl hängt von Arbeitsart, Stakeholder-Erwartungen, Team-Reife und Abhängigkeiten ab.

4.1 Faustregeln

	Scrum	Kanban
Arbeitsart	Produkt mit Iterationen, klare Sprint-Ziele	kontinuierliche Arbeit, wechselnde Prioritäten
Stakeholder	wollen Reviews in Sprint-Rhythmus	wollen schnelle Reaktion
Workload	einigermaßen planbar	viel Support, Unterbrechungen
Team-Reife	profitiert von Struktur	profitiert von Flexibilität
Abhängigkeiten	wenige innerhalb Sprint	flexibel pro Item

4.2 Hybrid-Modelle

In der Praxis nutzen viele Teams Mischformen:

- **Scrumban.** Scrum-Events (Planning, Review, Retro), aber Kanban-Board und WIP-Limits statt Sprint-Commitment.
- **Scrum + Kanban-Lane.** Klassischer Scrum-Backlog für Features, separate Kanban-Lane für Incidents/Anfragen mit WIP-Limit.

4.3 Entscheidung für ein konkretes Beispiel

Setting: Ein Unternehmen will in 12 Wochen einen digitalen Prozess (z. B. Rechnungsfreigabe) verbessern. Anforderungen ändern sich, Stakeholder sind viele, IT-Kapazität begrenzt.

Bewertung:

- **Risiko:** hoch (mehrere Schnittstellen, Compliance) → Struktur hilft.
- **Planbarkeit:** mittelmäßig (Anforderungen ändern sich).
- **Stakeholder:** viele, wollen Demos sehen.
- **Team-Reife:** mittel.
- **Workload:** Mix aus Features und Bugfixes/Anfragen.
- **Abhängigkeiten:** mehrere Systeme.

Empfehlung: Hybrid – Scrum für Features (2-Wochen-Sprints), Kanban-Lane für Bugfixes und Anfragen mit WIP-Limit 2.

4.4 Drei Policies, die Verantwortungsfähigkeit fördern

1. Klare Abnahme-Regel: *Erst wenn Product Owner “akzeptiert” geklickt hat, ist es Done.*
2. Stop-the-Line: *Wenn Quality Gate rot, niemand übernimmt neues Item, bis Gate grün ist.*
3. Transparenz: *Jedes Blocker wird sichtbar gemacht, mit Owner und Eskalations-Datum.*

5. Continuous Integration und Continuous Delivery

CI/CD ist die automatisierte Verwirklichung des Versprechens “schnelle, sichere Änderungen”. CI sorgt dafür, dass jeder Code-Commit automatisch gebaut und geprüft wird; CD sorgt dafür, dass Änderungen mit hoher Frequenz in höhere Umgebungen ausgerollt werden. (Humble & Farley, 2010; Forsgren et al., 2018)

5.1 Eine typische Pipeline in sechs Stufen

1. **Commit.** Code wird in Versionskontrolle eingecheckt.
2. **Build.** Quellcode wird kompiliert/gebündelt.
3. **Unit Tests.** Komponenten-Tests laufen.
4. **Integration Tests.** Tests gegen andere Services, Datenbanken, APIs.
5. **Security / Quality Checks.** Statische Analyse, Vulnerability-Scan, Coverage-Check.
6. **Deploy.** Staging, dann Production – mit Smoke Tests dazwischen.

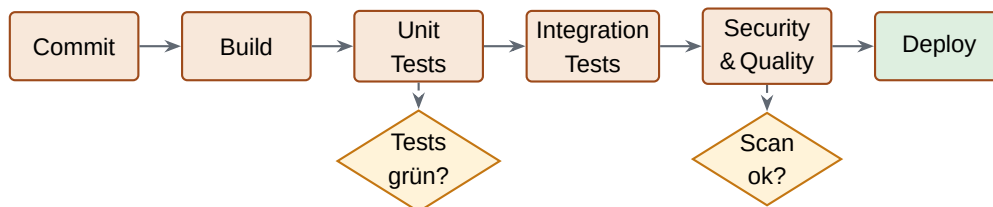


Abbildung 1: CI/CD-Pipeline – sechs Stufen mit zwei Pflicht-Quality-Gates.

5.2 Quality Gates

Ein **Quality Gate** ist eine Stop-Regel: Wenn eine Bedingung nicht erfüllt ist, geht die Pipeline nicht weiter. Zwei Gates sind in jeder produktiven Pipeline Pflicht:

- **Tests grün.** Unit- und Integration-Tests bestehen.
- **Security-Scan ok.** Keine kritischen Vulnerabilities, keine offenen Secrets im Code.

Weitere Gates (kontextabhängig):

- Code-Coverage über Schwelle.
- Statische Analyse ohne kritische Findings.
- Performance-Tests im Rahmen.
- Smoke Tests nach Deploy ok.

5.3 Deployment-Strategien

- **Blue/Green.** Schneller Rollback durch Switch zwischen Umgebungen.
- **Canary.** Schrittweise Aktivierung auf Prozent der Nutzer.
- **Rolling.** Schrittweise Ablösung der Instanzen.
- **Feature Toggle.** Funktion deployed, aber per Schalter ein/aus.

5.4 DORA-Metriken

Vier Kennzahlen aus dem “Accelerate”-Buch (Forsgren et al., 2018), die den Reifegrad von Liefer-Organisationen messen:

- **Lead Time for Changes.** Zeit von Code-Commit bis Production.
- **Deployment Frequency.** Wie oft wird in Production deployed?
- **Change Failure Rate.** Anteil Deployments, die zu Incident führen.
- **Mean Time to Restore (MTTR).** Wie schnell ist nach Incident wieder Service da?

DORA als Steuerungssprache

DORA-Metrik	“High Performer”	“Low Performer”
Lead Time for Changes	<1 Tag	>1 Monat
Deployment Frequency	mehrmals täglich	weniger als einmal pro Monat
Change Failure Rate	<15 %	>45 %
MTTR	<1 Stunde	>1 Woche

5.5 Anti-Gaming bei DORA

DORA-Metriken sind nicht immun gegen Gaming:

- Deployment Frequency hochtreiben durch leere Deploys → Verkoppeln mit Change Failure Rate.
- Change Failure Rate künstlich niedrig halten durch nicht-Reporting von Incidents → unabhängige Beobachtung (Monitoring-Alerts).
- MTTR senken durch “schließen ohne lösen” → Verkoppeln mit Reopen-Rate.

CI/CD ist nicht Tool, sondern Vertrag

Eine Pipeline ohne Quality Gates ist eine schnelle Möglichkeit, Defekte in Produktion zu bringen. Quality Gates ohne Konsequenz (wenn rot, deployt jemand trotzdem manuell) sind Theater. CI/CD trägt nur, wenn die Gates auch unter Druck verteidigt werden – das ist eine Führungs-Aufgabe, nicht eine Tool-Konfiguration.

6. Infrastructure as Code (IaC)

Infrastructure as Code bedeutet: Infrastruktur (Server, Netzwerke, Datenbanken, Konfigurationen) wird in versioniertem Code beschrieben – und nur über diesen Code geändert. Klick-Konfiguration ist tabu. (Morris, 2020)

6.1 Vier Kernprinzipien

- **Code statt Click.** Jede Infrastruktur-Änderung als Pull Request.
- **Versionierung.** Git oder ähnliches Repository als Single Source of Truth.
- **Review-Pflicht.** Vier Augen vor Merge in Hauptzweig.
- **Reproduzierbarkeit.** Aus dem Code wird jede Umgebung identisch erzeugt.

6.2 Typische Werkzeuge (toolneutral)

- *Terraform* – Cloud-übergreifend, deklarativ.
- *CloudFormation* / *Bicep* / *ARM* – Cloud-spezifisch.
- *Ansible* / *Puppet* / *Chef* – Konfigurations-Management.
- *Kubernetes Manifests* / *Helm* – für Container-Orchestrierung.

6.3 IaC-Policy in fünf Punkten

1. **Wer darf was?** – Klares Rollenmodell: wer schreibt, wer reviewt, wer merged.
2. **Pull Request Pflicht.** Keine direkten Pushes in Main.
3. **Mindestens 2-Augen-Review.** Für produktive Umgebungen.
4. **Secrets-Management.** Keine Secrets im Repo. Vault oder Cloud-Secrets-Service.
5. **Audit-Trail.** Merge-Historie als Audit-Log; Auswertbar pro Umgebung und Zeitraum.

6.4 Nutzen und Risiken

- **Nutzen:** Standardisierung, Audit-Fähigkeit, schnelle Umgebungen, Rollback per Git-Revert.
- **Risiken:** Falsche Rechte (jemand merged etwas Kritisches ohne Review), Secrets-Leaks (versehentliches Einchecken), unkontrollierte “Hot Fixes” direkt in der Umgebung, die später vom Code überschrieben werden.

Eselsbrücke: C-V-R-R

Vier IaC-Eigenschaften, die zusammen tragen:

Code statt Click.

Versionierung im Repo.

Review-Pflicht vor Merge.

Reproduzierbarkeit der Umgebungen.

Wer einen Buchstaben weglässt, baut Infrastruktur-Wildwuchs unter neuem Namen.

7. Fallstudie: CityServices – digitaler Genehmigungsworkflow

CityServices ist ein öffentlicher Service-Provider (Verwaltung). Ein Genehmigungsprozess dauert aktuell 6 Wochen, hat Medienbrüche, Bürger sind unzufrieden. Gleichzeitig hohe Compliance-Anforderungen, Releases sind selten und riskant.

7.1 Ausgangslage und Restriktionen

- Zeit: 10 Wochen bis Pilot.
- Budget: 130 000 €.
- Fachbereich will schnelle Änderungen, IT fürchtet Instabilität.
- Anforderungen sind unklar, ändern sich erfahrungsgemäß.

7.2 Schritt 1 – Scrum/Kanban-Entscheidung

Empfehlung: Hybrid.

- *Scrum* für neue Features in 2-Wochen-Sprints, mit Stakeholder-Demos.
- *Kanban-Lane* für Bugfixes und Bürger-Anfragen, WIP-Limit 2.

Operating Model

- Rollen: Product Owner (Fachbereich), Scrum Master (Service-Lead), Developers (Mixed Team Fach + IT).
- Events: Planning Mo, Daily 10 min Mo–Fr, Review Fr Woche 2, Retro im Anschluss.
- Backlog: priorisiert nach Wert für Bürger (Mess-Surrogat: “Bedeutung für Verfahrens-Beschleunigung”).
- Kanban-Lane Policies: Pull bei freier Kapazität, WIP-Limit 2, Eskalation an Service-Lead

bei >5 Tage Aging.

7.3 Schritt 2 – 12-Wochen-Plan in drei Releases

MVP (Woche 1–4)

- Digitaler Antrag online (Pflichtfelder, Vorab-Validierung).
- Statusanzeige für Bürger (“Mein Antrag wird gerade geprüft”).
- Ein Genehmigungsweg (häufigster Antragstyp).

Outcomes (nicht Tasks):

- Antrag in <10 min vom Bürger ausfüllbar.
- 100 % der Anträge laufen über das System (kein Paper-Backup).
- Statusanzeige aktualisiert in <1 h.

Beta (Woche 5–8)

- Rollenmodell (Antragsteller, Bearbeitende, Genehmigende, Audit).
- SLA-Reminder und Eskalation bei Fristüberschreitung.
- Audit Trail.
- Zweiter Genehmigungsweg.

Outcomes:

- Fristen werden in >90 % der Fälle eingehalten.
- Audit-Stichprobe besteht ohne Findings.
- Bürger-Eskalationen sinken um >50 %.

Stabilisierung (Woche 9–12)

- Performance-Optimierung.
- Monitoring + Alerts.
- Fehlerfall-Behandlung.
- Prozess-KPIs für Steuerung.

Outcomes:

- p95-Antwortzeit <2 s.
- MTTR im Pilot-Betrieb <2 h.
- KPI-Dashboard für Service-Lead live.

7.4 Schritt 3 – CI-Pipeline und Quality Gates

Pipeline:

- Commit → Build → Unit Tests → Integration Tests → Security Scan → Deploy Staging → Smoke Tests.

Quality Gates (nicht verhandelbar):

1. Tests grün (Unit + Integration).
2. Security Scan ohne kritische Findings.

Release-Policy:

- Deploys nach Production: Mo–Do, nicht Freitag.
- Emergency Releases: nur mit explizitem Approval des Service-Leads, mit Rollback-Plan.

7.5 Schritt 4 – IaC-Policy

- Pull-Request-Pflicht für alle Änderungen.
- 2-Augen-Freigabe für Produktiv-Umgebungen.
- Umgebungen reproduzierbar (Staging = Production minus Skalierung und Daten).
- Secrets nicht im Repo; zentrale Secret-Lösung.
- Audit-Log via Merge-Historie; monatlicher Audit-Report.

7.6 Schritt 5 – Fast vs. Safe-Entscheidungen

Bewusst Speed:

- MVP ohne vollständige Automatisierung aller Sonderfälle (manuelle Fallbacks akzeptiert).
- Performance-Optimierung erst in Stabilisierungs-Phase.

Bewusst Safety:

- Nicht verhandelbare Quality Gates (Security + Smoke).
- Rollback-Plan vor jedem Production-Deploy verpflichtend.
- Datenschutz-Review vor Live-Gang der Antragsdaten.

7.7 Annahmen und Frühwarnsignale

- **Annahme:** 2-Wochen-Sprints reichen für die geplanten Outcomes.
- **Annahme:** Pull-Request-Reviews schaffen einen Tag.
- **Frühwarnsignal 1:** Change Failure Rate >20 % in einer Woche → Quality Gates verschärfen.
- **Frühwarnsignal 2:** Lead Time steigt >5 Tage → WIP-Limit oder Replenishment prüfen.

7.8 Lessons aus der Fallstudie

- Hybrid kann ehrlicher sein als “rein Scrum” oder “rein Kanban”.
- Outcomes statt Tasks zwingen das Team zur Wirkungs-Perspektive.
- Quality Gates sind kein IT-Thema, sondern eine Führungsentscheidung.

8. Management-Perspektive: Agile als Steuerungssystem

Auf Wissens-Ebene sind Scrum, Kanban, CI/CD und IaC einzelne Methoden. Auf Anwendungsebene sind sie Handwerk. Auf Management-Ebene sind sie zusammen ein *Steuerungssystem für Veränderung*. (Highsmith, 2009; Forsgren et al., 2018)

8.1 Zwei zentrale Zielkonflikte

1. **Release-Frequenz vs. Risiko.** Mehr Releases pro Woche bedeuten kürzere Lead Times – und potenziell mehr Incidents, wenn die Gates schwach sind. Praxis: *Release-Frequenz schrittweise erhöhen, mit Change Failure Rate als Gate*. Wenn Failure Rate steigt → Frequenz reduzieren, Quality Gates verschärfen.
2. **Standardisierung vs. Team-Autonomie.** Zentrale CI/IaC-Standards (Pipelines, Quality Gates, Security) erhöhen Sicherheit – und reduzieren Team-Geschwindigkeit. Pure Team-Autonomie schafft Wildwuchs. Praktikabel: *harte Standards für Cross-Cutting (Security, Audit, IaC-Struktur), Autonomie bei Sprint-Inhalt, Tooling-Details, Implementations-Entscheidungen*.

8.2 Operating Model – wer verantwortet was

- **Product Owner.** Verantwortet Backlog-Inhalt, Prioritäten, Akzeptanz.
- **Scrum Master / Team Coach.** Verantwortet Lebensfähigkeit des Prozesses.
- **Platform Team.** Stellt CI/CD-Plattform und IaC-Standards bereit.
- **Security Team.** Definiert Security-Standards, betreibt Scans, verantwortet kritische Gates.
- **Service-Owner / Operations.** Verantwortet Production-Betrieb, MTTR.

8.3 Risiko-Akzeptanz-Modell

Eine zentrale Entscheidung: *Wer darf welches Risiko akzeptieren?* Beispiele:

- Production-Deploy am Freitag: nur Service-Lead, mit Rollback-Plan und Pager-Bereitschaft.
- Emergency-Release ohne vollständige Integration-Tests: Service-Lead + Head of Engineering.
- Bypass eines Quality Gates: nur CTO oder benanntes Risk-Komitee.

8.4 Stop-the-Line-Regel

Eine Senior-Disziplin: definiere klar, wann *niemand* mehr liefern darf. Typische Trigger:

- Security-Scan zeigt kritische Schwachstelle.
- Change Failure Rate >30 % in einer Woche.
- Wiederkehrende Incidents auf demselben Service.

In diesen Fällen wird das Team aus dem Sprint herausgenommen und konzentriert sich auf die Ursachen-Behebung. Diese Regel schriftlich, mit Owner und Eskalations-Pfad.

8.5 Vermeidung des “Agile = mehr Arbeit”-Falle

Eine häufige Fehlinterpretation: Agile bedeutet, dass das Team in derselben Zeit mehr liefert. Realistischer: Agile bedeutet, dass das Team in derselben Zeit *wirksamer* liefert – weniger Verschwendung, weniger Rework, kleinere Increments. Drei Maßnahmen:

- Outcomes statt Tasks im Backlog (was wird besser, nicht was wird getan).
- Definition of Done streng halten – weniger “halbfertige” Items.
- Retrospective mit *einer* konkreten Verbesserung pro Sprint, nicht zehn.

8.6 Entscheidungen unter Unsicherheit

- **Release-Frequenz-Ziel.** Wo wollen wir in 6 Monaten sein? Schriftlich, mit Trigger (Change Failure Rate über X → Frequenz reduzieren).
- **Standardisierung vs. Team-Autonomie.** Welche fünf Cross-Cutting-Themen sind nicht verhandelbar? Welche bleiben lokale Entscheidung?

Schluss-Merksatz für Tag 9

Scrum/Kanban sind Steuerungssysteme, keine Meeting-Kalender. CI/IaC sind Verträge, keine Tools. DORA-Metriken sind Sprache, kein Selbstzweck. Senior-Praxis verbindet alle drei zu einem System, in dem *Veränderung gesteuert, nicht erlitten* wird.

9. Take-aways aus Tag 9

-
1. **Scrum = Taktung, Kanban = Flow.** Hybrid ist legitim, oft ehrlicher als “rein”.
 2. **WIP-Limit ist Selbstdisziplin.** Ohne Disziplin nur Dekoration.
 3. **Outcomes statt Tasks.** Backlog-Items zeigen, was *besser* wird – nicht was *getan* wird.
 4. **Quality Gates nicht verhandelbar.** Mindestens Tests grün und Security ok – der Rest pro Kontext.
 5. **IaC = C-V-R-R.** Code, Versionierung, Reviews, Reproduzierbarkeit.
 6. **DORA als Sprache.** Lead Time, Deployment Frequency, Change Failure Rate, MTTR – alle vier mit Anti-Gaming.
 7. **Stop-the-Line-Regel.** Schriftlich, mit Owner und Eskalation. Niemand liefert, wenn Gate rot.
 8. **Fast vs. Safe schriftlich entscheiden.** Wo bewusst Speed, wo bewusst Safety – begründet, mit Frühwarnsignal.

10. Literatur

Im Skript verwendete und für die Vertiefung empfohlene Quellen. Zitierstil: APA-7.

Anderson, D. J. (2010).

Kanban: Successful evolutionary change for your technology business. Blue Hole Press.

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., ... & Thomas, D. (2001).

Manifesto for agile software development. <https://agilemanifesto.org/>

Forsgren, N., Humble, J., & Kim, G. (2018).

Accelerate: The science of lean software and DevOps – Building and scaling high performing technology organizations. IT Revolution Press.

Highsmith, J. (2009).

Agile project management: Creating innovative products (2nd ed.). Addison-Wesley.

Humble, J., & Farley, D. (2010).

Continuous delivery: Reliable software releases through build, test, and deployment automation. Addison-Wesley.

Morris, K. (2020).

Infrastructure as code: Dynamic systems for the cloud age (2nd ed.). O'Reilly.

Reinertsen, D. G. (2009).

The principles of product development flow: Second generation lean product development. Celeritas Publishing.

Schwaber, K., & Sutherland, J. (2020).

The Scrum Guide: The definitive guide to Scrum – The rules of the game. Scrum.org. <https://scrumguides.org/guide.html>

11. Glossar

Die wichtigsten Begriffe des Tages – kurz definiert, in alphabetischer Reihenfolge.

Aging WIP

Wie lange ist ein Backlog-Item bereits “In Progress”? Frühwarnsignal für Stuck-Items.

Backlog

Geordnete Liste aller bekannten Anforderungen, gepflegt vom Product Owner.

Blue/Green Deployment

Strategie mit zwei Umgebungen, Wechsel per Switch – schneller Rollback.

Canary Deployment

Strategie, bei der eine neue Version zuerst auf wenige Nutzer aktiviert wird.

Change Failure Rate

DORA-Metrik: Anteil der Deployments, die zu Incident führen.

CI/CD

Continuous Integration / Continuous Delivery. Automatisierter Build-, Test-, Deploy-Prozess.

Cycle Time

Zeit von “In Progress” bis “Done” im Kanban; teamintern.

Daily Scrum

Tägliche 15-Minuten-Synchronisation des Scrum-Teams.

Definition of Done (DoD)

Prüfbare Liste, ab wann ein Backlog-Item als fertig gilt.

Deployment Frequency

DORA-Metrik: Wie oft wird in Produktion deployed?

DevOps

Kultur und Praxis, die Entwicklung und Betrieb durch Automation und gemeinsame Verantwortung verbindet.

DORA-Metriken

Vier Kennzahlen aus dem “Accelerate”-Buch zur Messung des Liefer-Reifegrads.

Feature Toggle

Mechanismus, mit dem deployte Funktionen ein-/ausgeschaltet werden können.

Flow

Kontinuierliche Bewegung von Arbeit durch das System; zentrale Kanban-Idee.

Infrastructure as Code (IaC)

Versionierte, reviewbare, reproduzierbare Beschreibung der Infrastruktur.

Increment

Nutzbares Ergebnis am Ende eines Sprints, das die DoD erfüllt.

Kanban

Lean-basierte Flow-Methode mit Visualisierung, WIP-Limits und Pull-System.

Lead Time

Zeit von “in Backlog” bis “Done”; geschäftsrelevante Größe.

MTTR

Mean Time to Restore. DORA-Metrik: durchschnittliche Zeit bis Wiederherstellung nach Inci-

dent.

Product Backlog

Geordnete Liste aller bekannten Anforderungen in Scrum.

Product Owner

Scrum-Rolle, verantwortet Inhalt und Priorisierung des Backlogs.

Pull Request

Anforderung zur Übernahme von Code-Änderungen, mit Review-Pflicht.

Pull-System

Prinzip im Kanban: Arbeit wird gezogen, nicht zugeteilt.

Quality Gate

Stop-Regel in einer Pipeline: ohne Erfüllung kein Weiterlaufen.

Replenishment-Meeting

Treffen, bei dem "Ready"-Spalte im Kanban nachgefüllt wird.

Retrospective

Scrum-Event am Ende des Sprints zur Team-Reflexion und Verbesserung.

Rollback

Rücknahme einer Änderung in einen vorherigen Zustand.

Scrum Master

Scrum-Rolle, coacht Team und beseitigt Impediments.

Sprint

Zeitlich begrenzte Iteration in Scrum, typischerweise 1 bis 4 Wochen.

Sprint Goal

Übergeordnetes Ziel des Sprints, das das Team gemeinsam verfolgt.

Stop-the-Line

Regel, nach der Lieferung gestoppt wird, wenn ein kritisches Gate verletzt ist.

Throughput

Anzahl abgeschlossener Items pro Zeiteinheit im Kanban.

WIP-Limit

Maximale Anzahl gleichzeitiger Items in einer Spalte; schützt Flow und Qualität.